Putevi i stabla

Negativne cijene, ruksak, A* i minimalno razapinjujuće stablo

dr.sc.tech. Kristijan Poje, 2026.

Okvirno pitanje: kada možemo dobro izabrati idući korak, a da ne izračunamo cijenu svega nakon njega?

Graf, cijena puta i aditivnost

Graf $G = (V, E)$, $n = |V|$ vrhova, $m = |E|$ bridova, cijena brida $w(u \to v)$.

- **Aditivnost:** cijena puta = **zbroj** cijena njegovih bridova: $w(P) = w(e_1) + \ldots + w(e_k)$.
- **Posljedica 1 — optimalna podstruktura:** svaki dio najjeftinijeg puta i sam je najjeftiniji put (inače zamijeni taj dio jeftinijim i cijeli put pojeftini).
- **Posljedica 2 — Bellmanova jednadžba:** $d(s) = 0$, $d(v) = \min_{u \to v} [\, d(u) + w(u \to v) \,]$. To je **uvjet** koji točan odgovor zadovoljava, ne algoritam.
- **Posljedica 3 — relaksacija** je lokalni popravak jednadžbe za jedan brid: ako $d(u) + w(u \to v) < d(v)$, postavi $d(v) \leftarrow d(u) + w(u \to v)$, $prev(v) \leftarrow u$.
- **Posljedica 4 — teleskopiranje:** dodamo li svakom bridu $h(v) - h(u)$, svaki put od s do t promijeni se za istu konstantu $h(t) - h(s)$ → poredak puteva ostaje isti.
- **Podsjetnik (Dijkstra):** vrh s najmanjom poznatom cijenom proglasi konačnim, relaksiraj njegove bridove. $O(m \log n)$. Pretpostavka: **nijedan brid nema negativnu cijenu**.

Karta predavanja: svako pitanje rađa sljedeće

#	algoritam	problem	alat	što daje	što ne zna → zato sljedeće
1	Bellman–Ford	najjeftiniji put iz *s* kad bridovi mogu biti negativni	relaksacija svih bridova u rundama (DP po broju bridova)	cijene svih vrhova ili negativni ciklus	$O(n \cdot m)$ je skupo; ako graf nema ciklusa, runde su suvišne
2	DAG / ruksak	0/1 ruksak: podskup težine ≤ *W* najveće vrijednosti	stanja (predmeti, kg) → DAG → jedan prolaz topološkim redom	najdulji put = optimalan podskup	rješava za sve vrhove; često treba samo jedan cilj
3	A*	najjeftiniji put *s* → *t* za **jedan** cilj *t*	Dijkstra + procjena ostatka *h* (nagib bridova)	put do cilja uz manje proglašenih vrhova	i dalje "put iz jednog vrha"; ne povezuje **sve** vrhove
4	Prim	najlakši skup bridova koji povezuje sve vrhove	Dijkstrin kostur, ključ = sam brid; svojstvo reza	minimalno razapinjujuće stablo	ne jamči ništa o pojedinom putu → natrag na najkraće puteve

Zajednička nit: **aditivnost → Bellmanova jednadžba** (1–3) · ista relaksacija, drugi **redoslijed** (1 → 2 → 3) · **nagib** bridova: potencijal *h* (1) → procjena do cilja (3) · **isti kostur s redom**, drugi ključ (Dijkstra → 3 → 4). Usmjereni graf: 1–3 ("kojim putem"). Neusmjereni graf: 4 ("tko je s kim povezan").

Odjeljak 1 — Bellman–Ford

Pitanje: najjeftiniji put iz *s* kad bridovi smiju biti negativni?

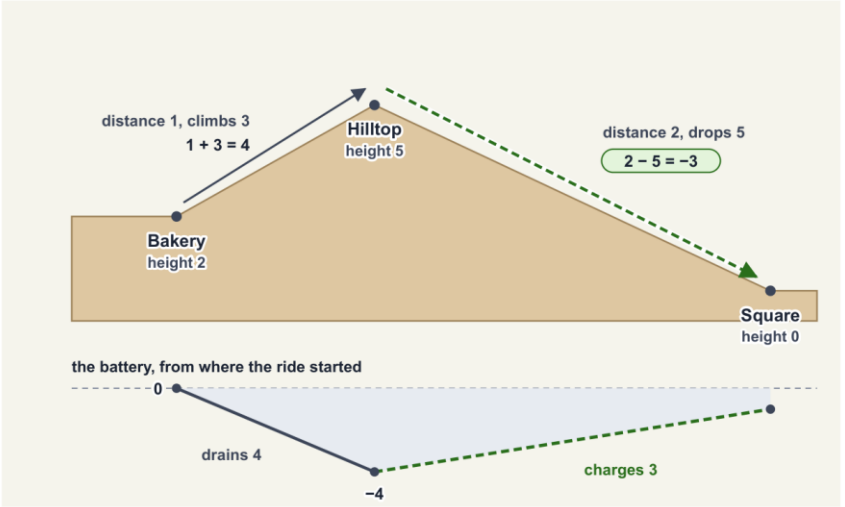
Zašto: Dijkstrin dokaz traži bridove ≥ 0 — jedan negativan brid ga ruši.

Novi sastojak: runde — nijedna cijena nije konačna dok se išta mijenja.

Negativne cijene bez negativnog ciklusa

Model: svaki vrh ima visinu $h(v)$; cijena brida = **udaljenost + $h(v)$ – $h(u)$** (energija s povratom pri spuštanju).

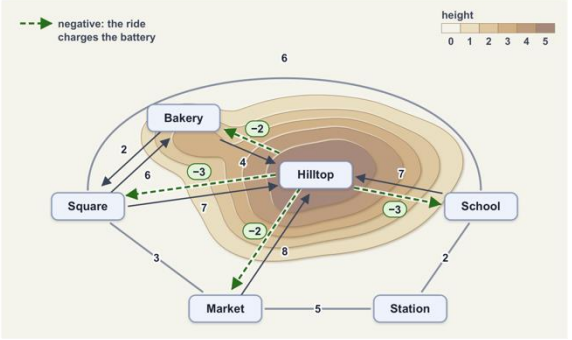
- B (h = 2) → H (h = 5), udaljenost 1: 1 + 3 = **4**.
- H → Sq (h = 0), udaljenost 2: 2 − 5 = **−3**.
- Brid između vrhova iste visine košta svoju udaljenost u oba smjera.
- Svaki ciklus se vraća na početnu visinu → razlike visina se poništavaju (aditivnost!) → cijena ciklusa = njegova udaljenost > 0. **Nema negativnog ciklusa**, ali ima **četiri negativna brida** (od 18).



ⓘ Try first (no AI)

Here are three places of Sara's town and three of its arrows, in energy (Figure 6.7): the Bakery to the Square costs 2, the Bakery to the Hilltop costs 4, and the Hilltop down to the Square costs −3. Your method from the last section settles the place with the smallest known cost first and never looks back. Run it from the Bakery. Say in one sentence whether you trust the cost it gives the Square.

Three minutes. You're done when you have the cost it gives the Square and the sentence.



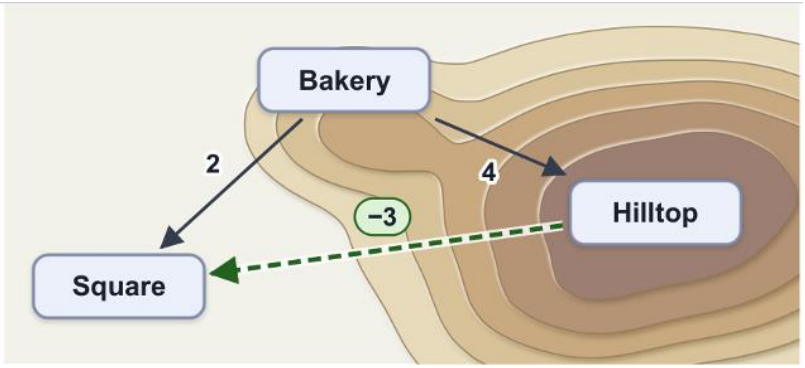
Dijkstra griješi s jednim negativnim bridom

Tri vrha i tri brida: B → Sq = 2, B → H = 4, H → Sq = −3.

korak	vrh koji se proglašava konačnim	poznate cijene
1	B, 0	Sq 2, H 4
2	**Sq, 2 — konačno**	H 4
3	H, 4	brid H → Sq se ne koristi: Sq je već konačan

Stvarno: B → H → Sq = 4 − 3 = **1 < 2**. Provjera Bellmanovom jednadžbom: $d(\text{Sq}) = \min(0 + 2, 4 - 3) = 1 \neq 2$ → Dijkstrin odgovor ne zadovoljava jednadžbu.

- Dokaz Dijkstre koristi: "svaki brid nakon prvog nekonačnog vrha košta ≥ 0". Jedan negativan brid ruši ga.
- Mjerenje (knjiga, cestovna mreža New Yorka s potencijalom po vrhu): 318 533 negativna brida → Dijkstra je jednako brz, ali **262 874 vrha** dobiju previsoku cijenu.
- Zaključak: **najmanja poznata cijena više ne smije biti konačna** — svaki kasniji brid još je može spustiti.



Ideja: dinamičko programiranje po broju bridova

$c_k(v)$ = najmanja cijena puta $s \rightsquigarrow v$ s **najviše *k* bridova**.

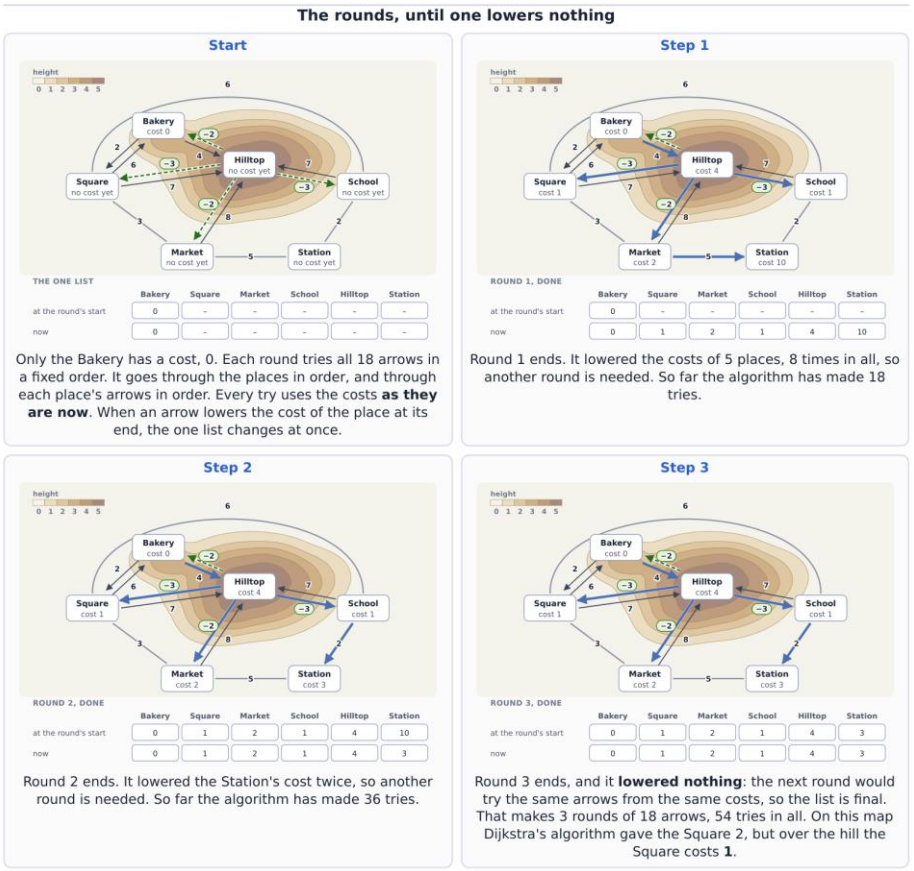
- $c_0(s) = 0$, $c_0(v) = \infty$ za $v \neq s$.
- **Rekurzija:** $c_k(v) = \min(c_{k-1}(v), \min_{u \to v} [c_{k-1}(u) + w(u \to v)])$ — Bellmanova jednadžba s brojačem *k*.
- Ako nema negativnog ciklusa dostupnog iz *s*, neki najjeftiniji put ne ponavlja vrh → ima ≤ $n - 1$ bridova → $\boldsymbol{c_{n-1} = d}$.
- **Bellman–Ford:** umjesto tablice drži **jednu listu** cijena; u svakoj **rundi** relaksira **sve** bridove fiksnim redom i mijenja listu odmah.
- Stop: runda u kojoj se ništa nije smanjilo (fiksna točka) — ili nakon $n - 1$ rundi + jedna provjera.

Runda 1 korak po korak

Redoslijed vrhova: B, Sq, M, Sch, H, St; bridovi svakog vrha redom (18 bridova). Početno: B = 0, ostali ∞.

brid	izračun	akcija
1. B → Sq	0 + 2 = 2	Sq ← 2
2. B → H	0 + 4 = 4	H ← 4
4. Sq → M	2 + 3 = 5	M ← 5
5. Sq → Sch	2 + 6 = 8	Sch ← 8
9. M → St	5 + 5 = 10	St ← 10
12. Sch → St	8 + 2 = 10	nije manje, bez promjene
14. H → Sq	4 − 3 = 1	Sq ← 1
15. H → M	4 − 2 = 2	M ← 2
16. H → Sch	4 − 3 = 1	Sch ← 1

Ostalih 9 bridova ništa ne mijenja. Nakon runde 1: **B 0 · Sq 1 · M 2 · Sch 1 · H 4 · St 10** — 8 smanjenja na 5 vrhova.



Runde 2 i 3, tablica naspram liste

- **Runda 2:** M → St: 2 + 5 = 7 → St ← 7; Sch → St: 1 + 2 = 3 → St ← 3. Ništa drugo.
- **Runda 3:** ništa se ne mijenja → **kraj**. Rezultat: B 0 · Sq 1 · M 2 · Sch 1 · H 4 · St 3.
- Rad: 3 runde × 18 bridova = **54 relaksacije**.

- **Invarijanta** (za bilo koji redoslijed bridova u rundi): nakon k rundi svaka cijena je (a) cijena nekog puta i (b) $\leq c_k(v)$. Iz (a) nikad ne padnemo ispod optimuma, iz (b) nakon $n - 1$ rundi smo na njemu.

k	B	Sq	M	Sch	H	St
0	0					
1	0	2			4	
2	0	1	2	1	4	
3	0	1	2	1	4	3

the table: at most k arrows

	B	Sq	M	Sc	H	St
k = 0	0					
k = 1	0	2			4	
k = 2	0	1	2	1	4	
k = 3	0	1	2	1	4	3

one list: after round r

	B	Sq	M	Sc	H	St
r = 0	0					
r = 1	0	**1**	**2**	**1**	4	**10**
r = 2	0	1	2	1	4	**3**
r = 3	0	1	2	1	4	3

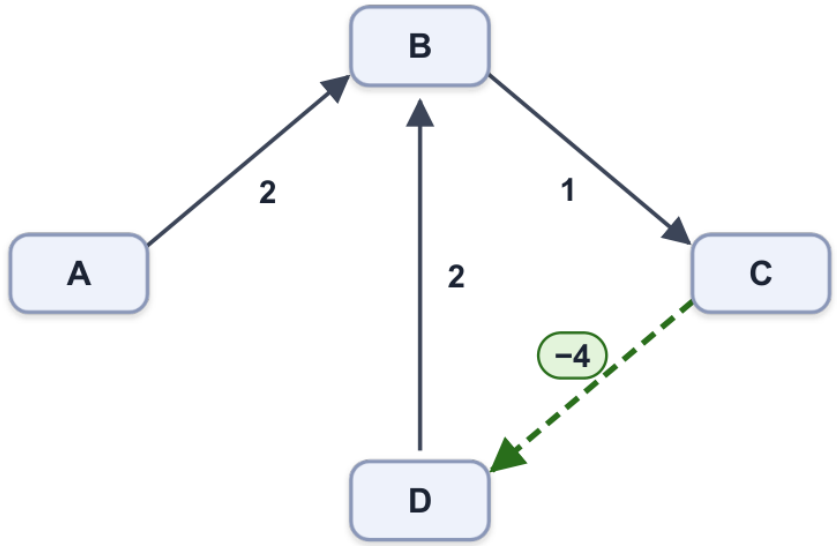
Negativni ciklus: detekcija i rekonstrukcija

Graf: A → B = 2, B → C = 1, C → D = −4, D → B = 2. Ciklus B → C → D → B: 1 − 4 + 2 = **−1**.

nakon runde	B	C	D
1	1	3	−1
2	0	2	−2
3	−1	1	−3
4 (= *n*)	**još se smanjuje**		

- Ako se cijena smanji i u *n*-toj rundi → iz *s* je dostupan **negativni ciklus**; najjeftiniji put do B, C, D **ne postoji** (aditivnost: svaki krug je za 1 jeftiniji).
- **Rekonstrukcija:** od zadnjeg smanjenog vrha napravi *n* koraka unatrag po *prev* → sigurno si na ciklusu; nastavi unatrag dok se ne vratiš u isti vrh → D → B → C → D (isti ciklus kao B → C → D → B).
- **Složenost:** O(*n* · *m*) vremena, O(*n*) memorije. Dijkstra: O(*m* log *n*).

graf (bez negativnih bridova)	runde	vrijeme BF / Dijkstra
slučajni, 100 000 vrhova, 400 000 bridova	15	1,25 ×
mreža, 90 000 križanja	62	5,3 ×
cestovna mreža New Yorka	614	65 ×



Vježba: pet vrhova, nepovoljan redoslijed bridova

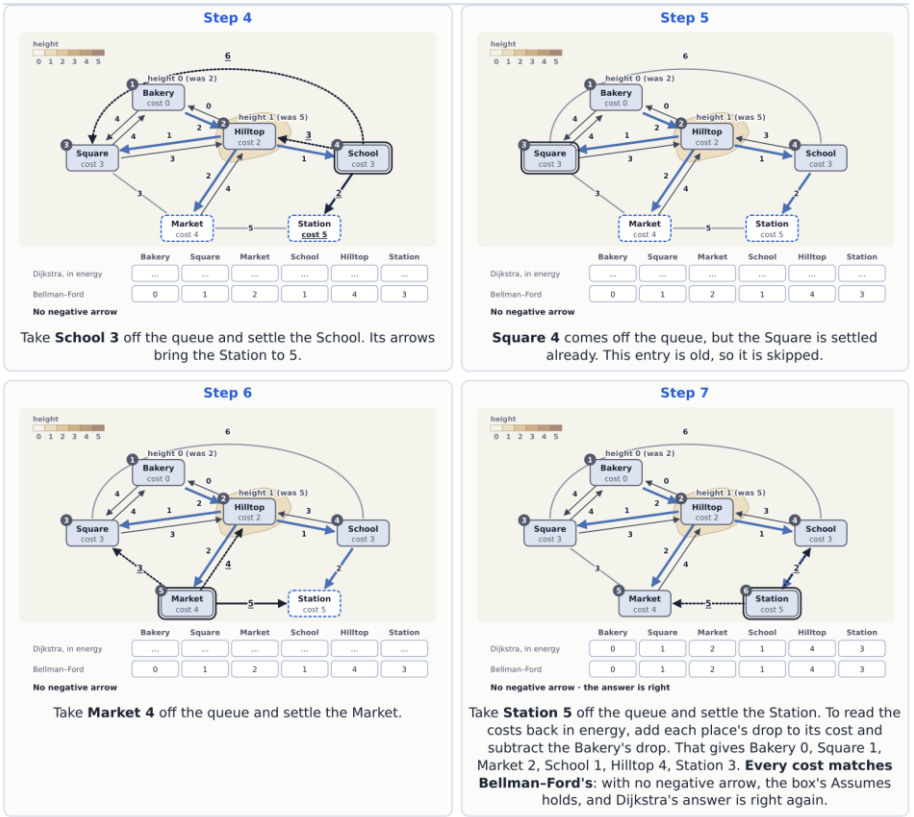
Vrhovi S, A, B, C, D ($n = 5$ → najviše 4 runde). Bridovi se u svakoj rundi obrađuju **ovim redom**: C → D 2 · A → C 3 · B → A **−3** · S → A 2 · S → B 4 · B → D 6.

nakon runde	**S**	**A**	**B**	**C**	**D**
0	0	∞	∞	∞	∞
1	0	2	4	∞	10
2	0	**1**	4	5	10
3	0	1	4	4	7
4	0	1	4	4	**6**

Provjera (runda 5): ništa se ne mijenja → nema negativnog ciklusa. Put do D po *prev*: S → B → A → C → D = 4 − 3 + 3 + 2 = **6**.

Kako zaobići runde: nagib i graf bez ciklusa

- **Nagib (potencijali):** $w'(u \to v) = w(u \to v) + h(u) - h(v)$. Po posljedici 4 (teleskopiranje) svi putevi pomaknu se za istu konstantu → poredak isti. Uz h = visina sa slajda 5: w' = udaljenost ≥ 0 → **Dijkstra opet radi**. (Johnson: h nađi jednim Bellman–Fordom — vježba.)
- **Graf bez ciklusa (DAG):** graf iz vježbe nema ciklusa → postoji **topološki poredak** S, B, A, C, D u kojem svaki brid ide unaprijed → obradi vrhove tim redom, svaki brid **jednom** → gotovo, O(n + m).
- Primjene Bellman–Forda: usmjerivači (RIP, distance-vector) — svaki zna samo susjede i ponavlja relaksaciju; Babel dodaje uvjet protiv petlji.
- Ograničenje: **negativni ciklus** se ne može "popraviti" — samo prijaviti.



Odjeljak 2 — Ruksak kao najdulji put u DAG-u

Pitanje: koji podskup predmeta težine ≤ *W* ima najveću vrijednost?

Zašto: Bellman–Ford radi runde samo zbog ciklusa; graf odluka nema ciklusa → dovoljan je jedan prolaz.

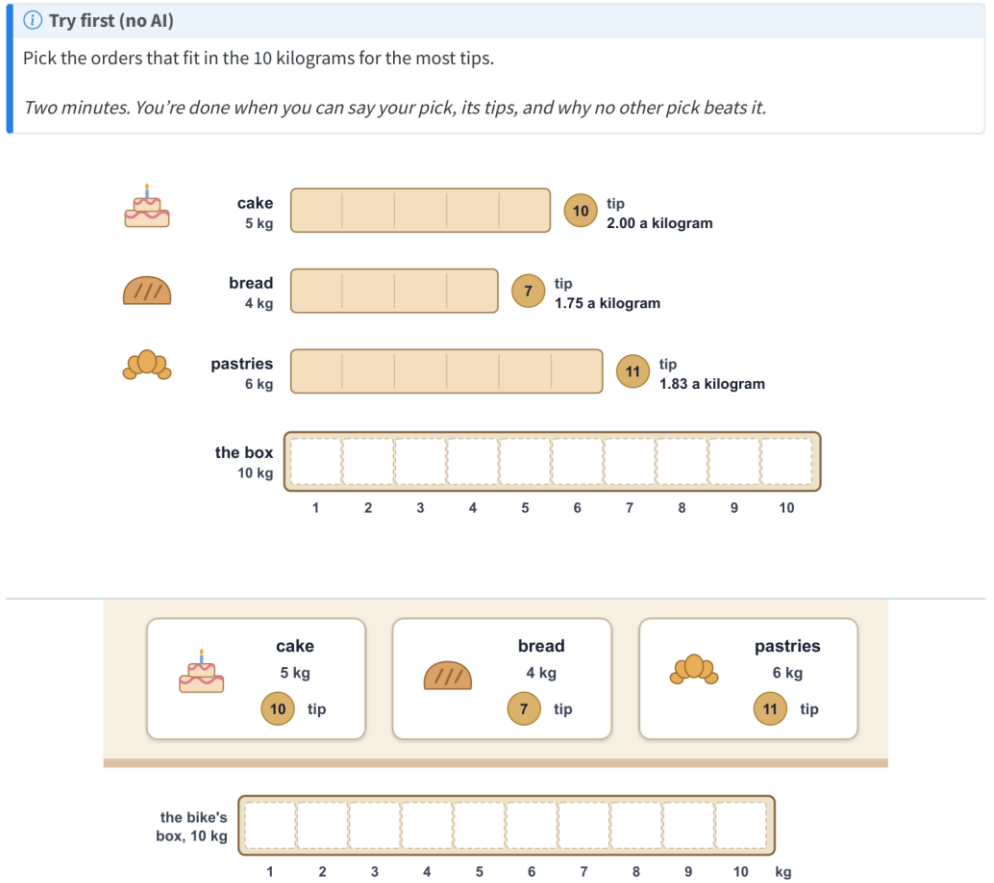
Novi sastojak: graf stanja koji gradimo iz samog problema.

Problem ruksaka i pohlepa koja ne radi

Kapacitet W = **10 kg**. Tri predmeta (primjer iz knjige):

predmet	težina	vrijednost	vrijednost / kg
torta	5 kg	10	2,00
kruh	4 kg	7	1,75
peciva	6 kg	11	1,83

- **0/1 ruksak:** izaberi podskup s ukupnom težinom ≤ W i najvećim zbrojem vrijednosti (vrijednost je **aditivna**).
- **Pohlepa po omjeru:** torta (5 kg) → peciva ne stanu → kruh: 9 kg, **17**.
- **Optimum:** kruh + peciva = 10 kg, **18**.
- Svih podskupova: 2^n — s 30 predmeta već ~10^9.



Od stabla odluka do DAG-a: spajanje istih stanja

Mali primjer: W = **3 kg**; X (1 kg, 3), Y (1 kg, 2), Z (2 kg, 4). Čvor = (kg potrošeno, vrijednost dosad).

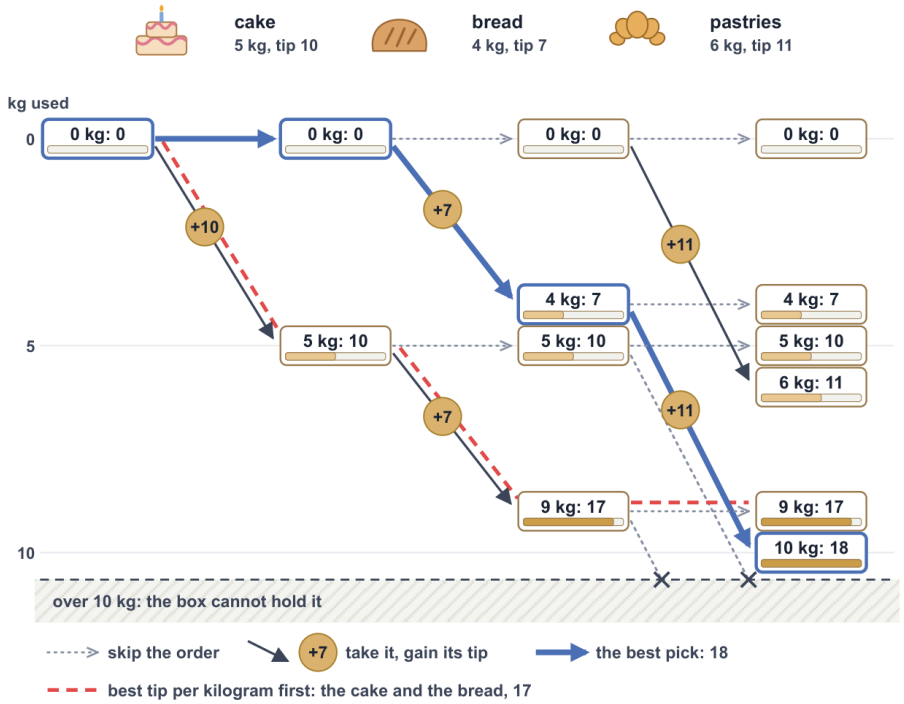
```
start (0 kg, 0)
├─ preskoči X → (0, 0)
│    ├─ preskoči Y → (0, 0) ── preskoči Z → (0, 0) · uzmi Z → (2, 4)
│    └─ uzmi Y     → (1, 2) ── preskoči Z → (1, 2) · uzmi Z → (3, 6)
└─ uzmi X → (1, 3)
     ├─ preskoči Y → (1, 3) ── preskoči Z → (1, 3) · uzmi Z → (3, 7) ← najbolje
     └─ uzmi Y     → (2, 5) ── preskoči Z → (2, 5) · uzmi Z → 4 kg  X
```

- Nakon X i Y: "preskoči X, uzmi Y" i "uzmi X, preskoči Y" obje imaju **1 kg** i ostaje samo Z → **isti skup mogućih nastavaka** → zadrži bolju (3), spoji čvorove.
- Nakon Z: 2 kg (4 ili 5 → zadrži 5), 3 kg (6 ili 7 → zadrži 7).
- **Spajanje istih stanja pretvara stablo (2^n listova) u DAG ($\leq n \cdot (W + 1)$ vrhova). To spajanje je dinamičko programiranje.**

Redukcija: stanja, bridovi, najdulji put

- **Vrh** (i, w): odlučeno o prvih i predmeta, potrošeno w kg. Start (0, 0).
- **Brid = jedna odluka o jednom predmetu**, iz sloja i u sloj $i + 1$:
  - **preskoči**: → ($i + 1$, w), vrijednost brida 0;
  - **uzmi** (ako $w + w_i \leq W$): → ($i + 1$, $w + w_i$), vrijednost brida v_i.

ruksak	**graf**
stanje (odlučeno, kg)	vrh
odluka uzmi / preskoči	brid
vrijednost predmeta	broj na bridu "uzmi" (preskoči = 0)
težina predmeta	**kamo** brid vodi (koliko stupaca dalje)
kapacitet W	bridovi koji bi prešli W ne crtaju se
jedan izbor	jedan put od starta do zadnjeg sloja
najbolji izbor	**najdulji put**



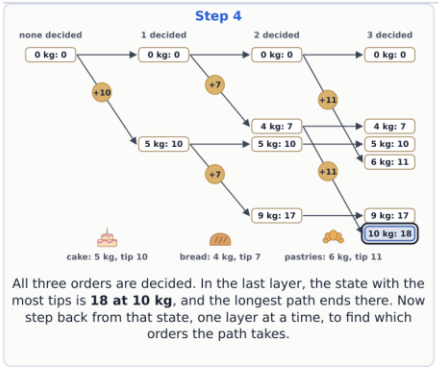
- Svaki brid ide u idući sloj → **DAG** → najdulji put je dobro definiran i lak. (U općem grafu najdulji jednostavni put je NP-težak.)
- Rekurzija: $B_i(w) = \max(\, B_{i-1}(w),\, B_{i-1}(w - w_i) + v_i \,)$ — Bellmanova jednadžba s max, najviše dva ulazna brida po vrhu.

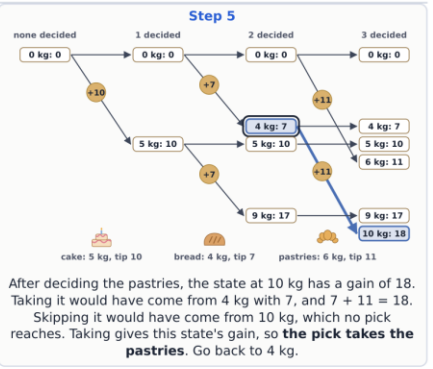
Punjenje slojeva korak po korak i povratak do izbora

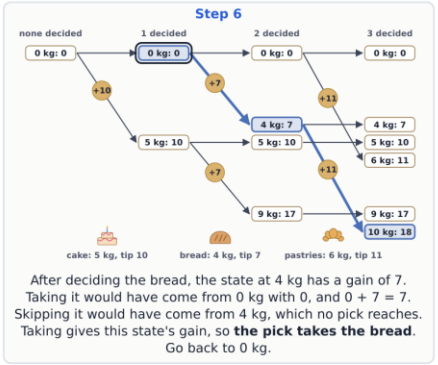
odlučeno	0 kg	4 kg	5 kg	6 kg	9 kg	10 kg
ništa	0					
+ torta (5, 10)	0		10			
+ kruh (4, 7)	0	7	10		17	
+ peciva (6, 11)	0	7	10	11	17	**18**

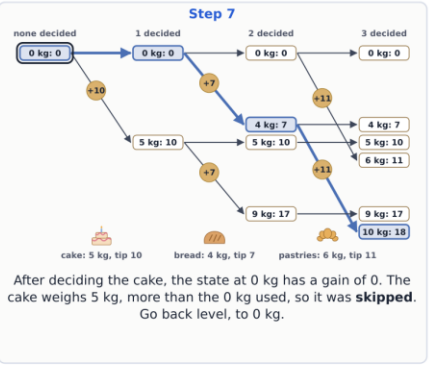
1. Sloj 1: iz (0, 0) preskoči → 0 kg: 0; uzmi → 5 kg: 10.
2. Sloj 2: iz 0 kg → 0 i 4 kg: 7; iz 5 kg → 5 i 9 kg: 17.
3. Sloj 3: iz 0 → 0 i 6: 11; iz 4 → 4 i **10: 18**; iz 5 i 9 peciva ne stanu (11 i 15 kg).
4. **Unatrag** od (3, 10): $B_2(4) + 11 = 18$ → peciva uzeta → (2, 4): $B_1(0) + 7 = 7$ → kruh uzet → (1, 0): torta ne stane → preskočena. (Pravilo iz knjige: ako uzmi i preskoči daju istu vrijednost, predmet se uzima.)

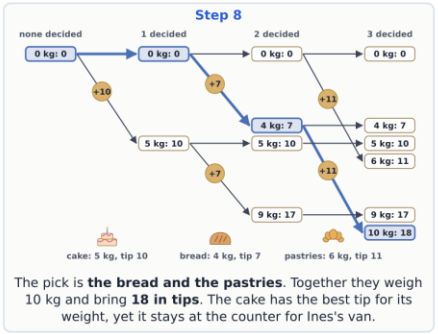
Izbor: **kruh + peciva, 10 kg, 18**.











Od Bellman–Forda do DAG-a: isti brid, drugi redoslijed

	Bellman–Ford	**DAG u topološkom redu**
redoslijed	svi bridovi, u rundama	vrhovi topološki, svaki brid jednom
broj prolaza	do $n - 1$ + provjera	1
zašto je dosta	invarijanta po rundama	svi ulazni bridovi vrha obrađeni prije njega
negativni bridovi	da	da
ciklusi	dopušteni (negativni → prijavi)	nisu dopušteni
min / max	min	min ili max (najdulji put = negiraj cijene)
složenost	$O(n \cdot m)$	$O(n + m)$

- Vježba sa slajda 11 u poretku S, B, A, C, D: A = min(2, 4 − 3) = 1, C = 4, D = min(10, 6) = 6 — **isti rezultat u jednom prolazu**.
- **Ruksak:** $n \cdot (W + 1)$ stanja, 2 brida po stanju → **$O(n \cdot W)$** — **pseudopolinomno** (polinomno u vrijednosti W, eksponencijalno u broju bitova od W). $W = 10$ je lako, težine u gramima do 10^9 nisu.
- Isti potez drugdje: udaljenost uređivanja, poravnanje DNA, `git diff` — put kroz mrežu slova.

Odjeljak 3 — Bellmanova jednadžba

Pitanje: što sve dosadašnje metode zapravo rješavaju?

Zašto: Dijkstra, Bellman–Ford i DAG rješavaju istu jednadžbu, samo drugim redoslijedom — i ništa ne znaju o ostatku puta.

Novi sastojak: jednadžba čitana od cilja — preostala cijena $r(v)$.

Jedna jednadžba, različiti redoslijedi; čitana od cilja

- **Unaprijed:** $d(s) = 0$, $d(v) = \min_{u \to v} [\, d(u) + w(u \to v) \,]$.
- **Od cilja:** $r(t) = 0$, $r(u) = \min_{u \to v} [\, w(u \to v) + r(v) \,]$ — **cijena koja je još preostala**.
- Svaki korak = **idući brid** + **ostatak puta**. Metode se razlikuju po tome što znaju o drugom članu.

metoda	**redoslijed rješavanja**	**što zna o ostatku *r***	**treba**
Dijkstra	najmanja poznata cijena prva	samo da je ≥ 0	nijedan brid < 0
Bellman–Ford	svi bridovi u rundama	ništa	nema negativnog ciklusa
DAG / ruksak	topološki	ništa	nema ciklusa
A*	najmanja cijena **+ procjena ostatka**	**donja granica $h \le r$**	*h* konzistentna

Provjera na vježbi sa slajda 11: $d(\mathrm{A}) = \min(0 + 2, 4 - 3) = 1$ ✓ · $d(\mathrm{D}) = \min(4 + 2, 4 + 6) = 6$ ✓.

Odjeljak 4 — A*

Pitanje: kako do jednog cilja *t* bez proglašavanja svih vrhova?

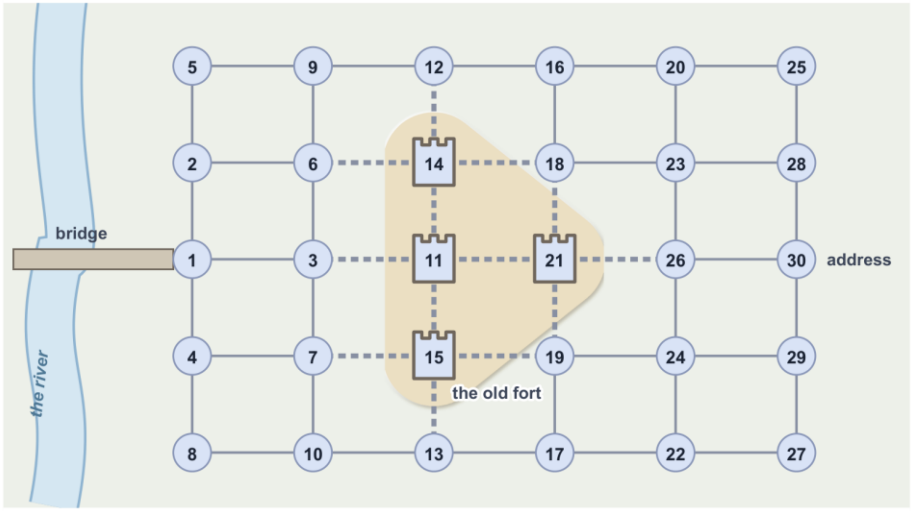
Zašto: Dijkstra, Bellman–Ford i DAG rješavaju za sve vrhove; donja granica ostatka usmjerava pretragu.

Novi sastojak: procjena *h* — nagib bridova iz Odjeljka 1, s procjenom umjesto visine.

Jedan cilj, a Dijkstra pretražuje u svim smjerovima

Mreža 6 × 5 križanja (stupci A–F, redovi 1–5). Start **A3**, cilj **F3**.

- Blok = **2 min**; blok koji **ulazi** u jedan od 4 posebna vrha (**C2, C3, C4, D3**) = **5 min**.
- Dijkstra iz A3 proglašava vrhove po cijeni — cijena raste jednako u svim smjerovima.
- Rezultat: proglašava **svih 30** vrhova, cilj je **zadnji**, 16 min.
- Ali: do cilja ima barem *k* blokova (Manhattan), a nijedan blok nije kraći od 2 min → **2 · (Manhattan udaljenost) je donja granica ostatka**.



A*: definicija i uvjet na procjenu

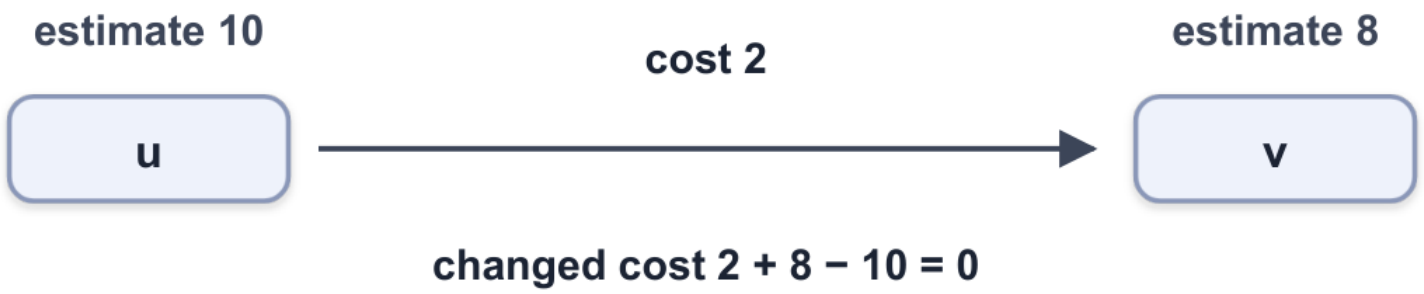
Ključ u prioritetnom redu: $f(v) = g(v) + h(v)$ — g poznata cijena od starta, h procjena ostatka. **Stani kad se cilj izvadi iz reda** (ne kad se ubaci).

- **Dopustiva** (admissible): $h(v) \le$ stvarni ostatak $r(v)$; $h(\text{cilj}) = 0$.
- **Konzistentna** (monotona): $h(u) \le w(u \to v) + h(v)$ za svaki brid — procjena po bridu pada najviše za cijenu brida. Konzistentna ⇒ dopustiva.
- $h \equiv 0$ je konzistentna → **A* = Dijkstra**.
- Ovdje: $h(v) = 2 \cdot (|\text{stupac}(v) - \text{F}| + |\text{red}(v) - 3|)$ — konzistentna jer svaki blok traje ≥ 2 min i mijenja Manhattan udaljenost za točno 1.

	Dijkstra	**A***
ključ	$g(v)$	$g(v) + h(v)$
stani	red prazan	cilj izvađen
daje	cijene svih vrhova	put do jednog cilja
treba	bridovi ≥ 0	bridovi ≥ 0 i h konzistentna

Zašto je točno: A* je Dijkstra na promijenjenim cijenama

- **Promijenjena cijena brida:** $w'(u \to v) = w(u \to v) + h(v) - h(u)$ — nagib sa slajda 12, s h umjesto visine.
- Konzistentnost $\Leftrightarrow w' \geq 0 \to$ **Dijkstra na *w'* je točan**.
- **Teleskopiranje (aditivnost):** $w'(P) = w(P) + h(\text{cilj}) - h(s) = w(P) - h(s) \to$ **svi putevi pomaknuti za istu konstantu**, najjeftiniji ostaje najjeftiniji.
- Dijkstrin ključ na $w' = g(v) + h(v) - h(s) \to$ isti poredak kao $f(v)$. **To je A*.**
- Jedan brid: $w = 2$, $h(u) = 10$, $h(v) = 8 \to w' =$ **0**: brid ravno prema cilju ne košta "ništa dodatno"; brid u stranu košta više.

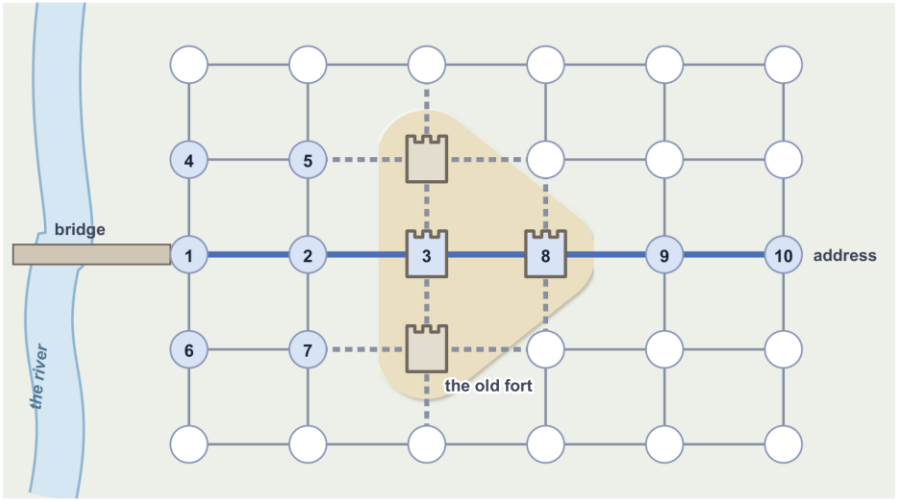


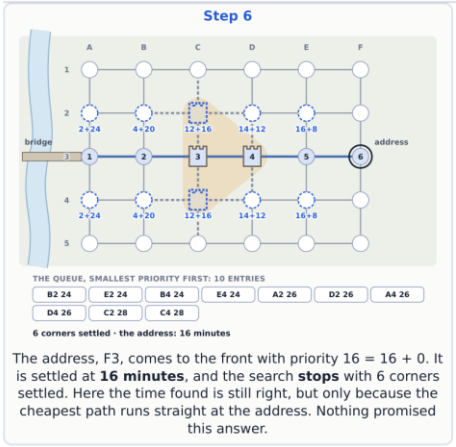
A* na mreži korak po korak

$f = g + h$, $h = 2 \cdot$ blokovi do F3.

korak	izvađeno iz reda	*g*	*h*	*f*	napomena
1	A3	0	10	10	dodaj B3 (2+8), A2 (2+12), A4 (2+12)
2	B3	2	8	10	dodaj C3 (7+6) — spori blok
3	C3	7	6	13	dodaj D3 (12+4), C2, C4 (12+8)
4–7	A2, B2, A4, B4	2, 4, 2, 4		14	traže zaobilazak; C2, C4 sad 9+8 = 17
8	D3	12	4	16	dodaj E3 (14+2)
9	E3	14	2	16	dodaj F3 (16+0)
10	**F3**	**16**	0	16	cilj — **stop**

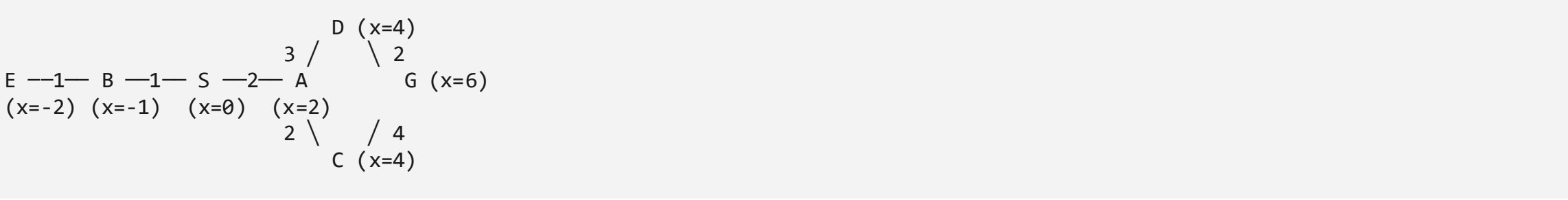
Rezultat: **16 min, 10 proglašenih vrhova umjesto 30**, put ravno kroz spore vrhove.





Vježba: zamka, "stani pri vađenju" i loša procjena

Neusmjereni graf (x = položaj na pravcu, cilj G na $x = 6$):



$h(v) = 6 - x$: E 8 · B 7 · S 6 · A 4 · C 2 · D 2 · G 0 (konzistentna: svaki brid ≥ promjena x).

korak	**izvadi ($g + h$)**	**dodaj / popravi**
1	S 0 + 6 = 6	A 2+4 = 6 · B 1+7 = 8
2	A 2 + 4 = 6	C 4+2 = 6 · D 5+2 = 7
3	C 4 + 2 = 6	G 8+0 = 8
4	D 5 + 2 = 7	**G 7+0 = 7** (bolje od 8)
5	**G 7**	**stop** — put S → A → D → G

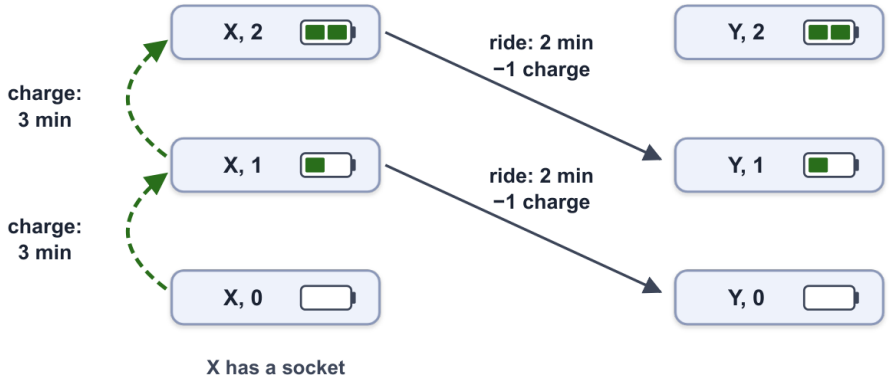
Dijkstra na istom grafu proglašava svih **7** vrhova (i B, E iza starta); A* **5**.

Kvaliteta procjene, proširenо stanje, ograničenja

- Uz konzistentnu *h* (i isto razbijanje izjednačenja) A* **nikad ne proglašava više vrhova** od Dijkstre. Što je *h* bliža *r*, to manje.
- Mjerenja iz knjige (oba algoritma staju na cilju):

graf	procjena	vrhova (A* / Dijkstra)	vrijeme (A* / Dijkstra)
New York, 100 slučajnih parova	zračna udaljenost	44 %	55 % (4,1 ms / 7,4 ms)
mreža 10^6 vrhova, svi bridovi 1	Manhattan	22 %	17 %
ista mreža, bridovi 1–1000	Manhattan × najjeftiniji brid	≈ 100 %	**105 %**

- **Prošireno stanje:** resurs s granicom (baterija) → vrh = (križanje, razina); punjenje = brid (*v*, *b*) → (*v*, *b* + 1). Isti A* na većem grafu — **isti potez kao kg kod ruksaka**.
- Ograničenja: **jedan cilj** (za sve vrhove → Dijkstra); procjena koja precjenjuje daje skuplji put; graf koji se mijenja tijekom izvođenja → D* Lite.



Odjeljak 5 — Minimalno razapinjujuće stablo: Primov algoritam

Pitanje: koji najlakši skup bridova povezuje sve vrhove?

Zašto: dosad „put iz jednog vrha”; sada ne pitamo kamo, nego tko je s kim povezan (neusmjereni graf).

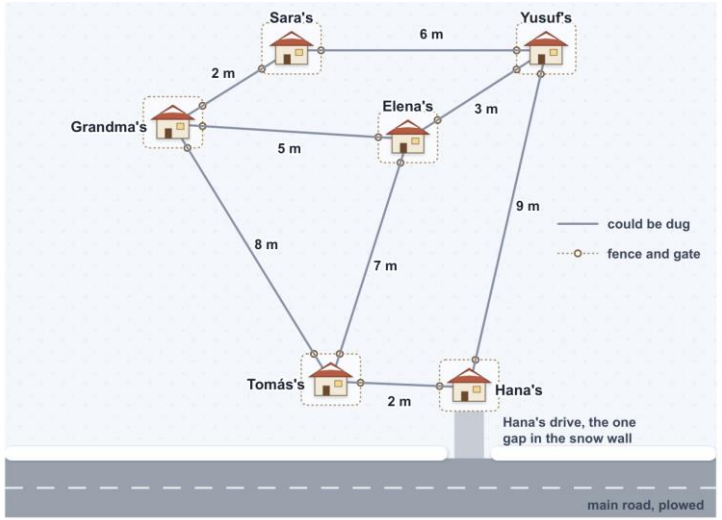
Novi sastojak: ključ = sam brid; dokaz = svojstvo reza.

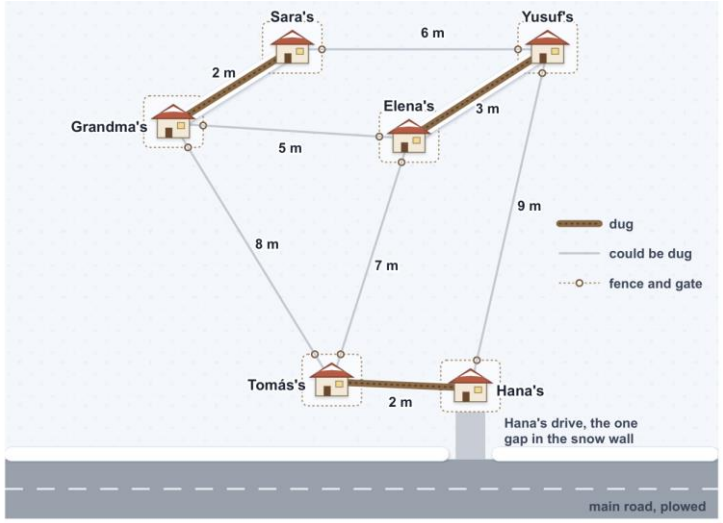
Problem: minimalno razapinjujuće stablo

Šest vrhova (oznake iz knjige): G, S, Y, E, T, H.
Neusmjereni bridovi (težine):

G–S 2 · T–H 2 · Y–E 3 · G–E 5 · S–Y 6 · E–T 7 · G–T 8 · Y–H 9

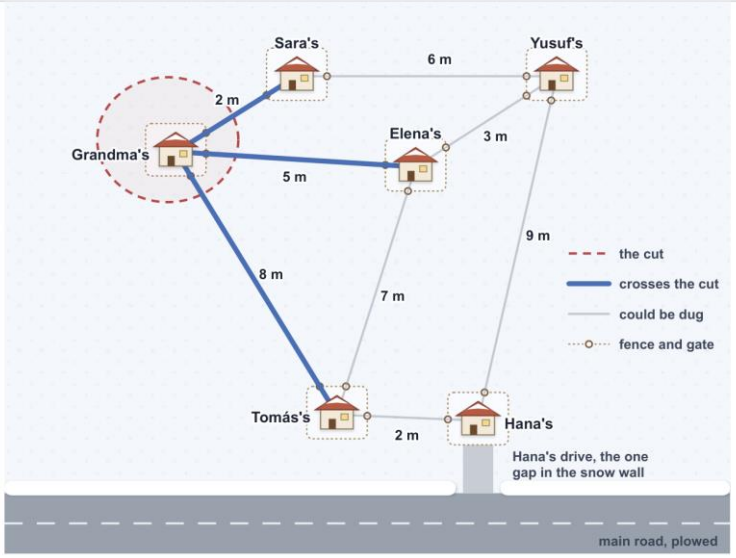
- **Razapinjujuće stablo** povezanog grafa: skup bridova koji povezuje sve vrhove i ne zatvara ciklus; ima ***n* – 1** bridova.
- **Minimalno razapinjujuće stablo (MST):** razapinjujuće stablo najmanjeg zbroja težina.
- Lokalna pohlepa "svaki vrh uzme najlakši svoj brid": G–S, Y–E, T–H = 7 → **tri komponente**, graf nije povezan.





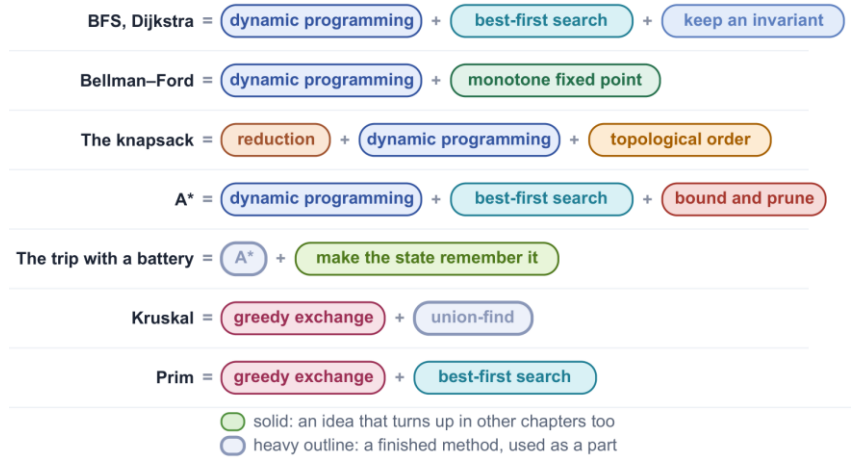
Svojstvo reza

- **Rez** = podjela vrhova na dvije neprazne strane (S, $V \setminus S$). Brid **prelazi rez** ako ima kraj na svakoj strani.
- **Svojstvo reza:** najlakši brid koji prelazi bilo koji rez nalazi se u **nekom** MST-u.
- **Dokaz (zamjena):** neka MST T ne sadrži najlakši brid e preko reza. $T + e$ ima točno jedan ciklus; ciklus prelazi rez paran broj puta → postoji drugi brid f na ciklusu preko reza, $w(f) \geq w(e)$. $T + e - f$ je razapinjujuće stablo težine $\leq w(T)$. ■
- Primjer: rez {G} | ostali → prelaze G–S 2, G–E 5, G–T 8 → **G–S je siguran**.



Od Dijkstre do Prima: ista petlja, drugi ključ

	Dijkstra	A*	Prim
ključ vrha *v* (preko brida *u*–*v*)	$g(u) + w$	$g(u) + w + h(v)$	**samo *w***
aditivnost puta	da	da	**ne**
stani	red prazan	cilj izvađen	red prazan
daje	stablo najkraćih puteva	put do cilja	MST
zašto točno	bridovi ≥ 0	*h* konzistentna → $w' \geq 0$	svojstvo reza
graf	usmjeren	usmjeren	neusmjeren



- **Prim:** raste **jedno stablo** od jednog vrha; u svakom koraku dodaj najlakši brid između stabla i ostatka (rez = stablo | ostali). Stari zapisi u redu se preskaču, kao kod Dijkstre.

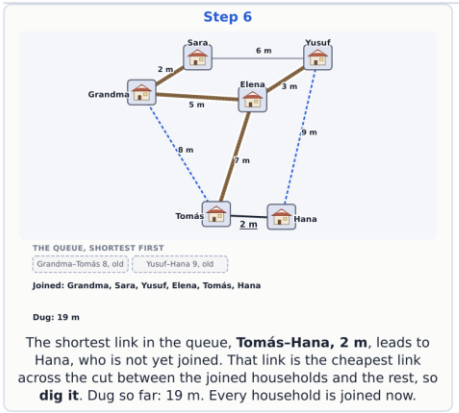
Primov algoritam korak po korak

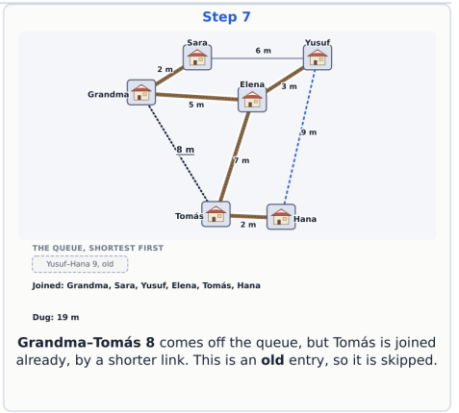
Ključ = **težina brida**. Start: G.

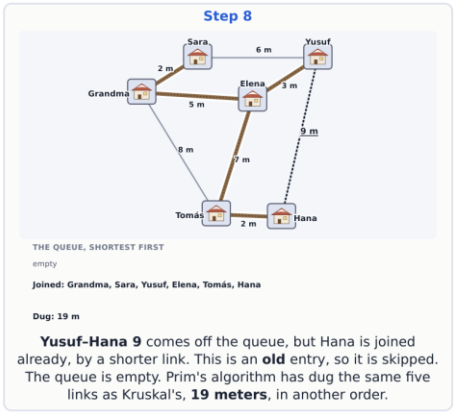
korak	izvađeno iz reda	dodano	nove / bolje ponude
0	—	G	S 2, E 5, T 8
1	S 2	G–S	Y 6
2	E 5	G–E	Y 3 (bolje od 6), T 7 (bolje od 8)
3	Y 3	E–Y	H 9
4	Y 6	—	stari zapis, Y već u stablu → preskoči
5	T 7	E–T	H 2 (bolje od 9)
6	H 2	T–H	—
7	T 8, H 9	—	stari zapisi → preskoči; red prazan

MST: G–S 2, G–E 5, E–Y 3, E–T 7, T–H 2 = **19**.

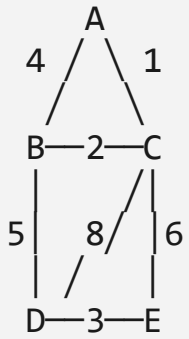
- **Provjera Kruskalom:** bridovi od najlakšeg, preskoči onaj koji zatvara ciklus (union-find iz 5. tjedna): G–S, T–H, Y–E, G–E, S–Y ✗, E–T → istih 5 bridova, 19, drugim redoslijedom; dva najteža brida Kruskal ni ne pogleda.
- **S Dijkstrinim ključem** iz G: G–S, S–Y, G–E, G–T, T–H = **23** — stablo najkraćih puteva, ne MST.







Vježba: pet vrhova, Prim naspram Dijkstre



korak	izvadi	dodaj	nove / bolje ponude
0	—	A	B 4, C 1
1	C 1	A–C	B 2 (bolje od 4) · D 8 · E 6
2	B 2	C–B	D 5 (bolje od 8)
3	B 4	—	stari zapis → preskoči
4	D 5	B–D	E 3 (bolje od 6)
5	E 3	D–E	gotovo

MST = 1 + 2 + 5 + 3 = 11. Dijkstra iz A: A–C, C–B, C–E, B–D = **14**.

Složenost, zašto pohlepa radi

- **Složenost:** binarna gomila **O(*m* log *n*)** — rijetki grafovi; polje ključeva **O(n^2)** — gusti grafovi ($m \approx n^2$). Knjiga: potpuni graf, 5000 točaka (12,5 mil. bridova) → polje 0,54 × vremena gomile; 4000 točaka, 10 % parova (~800 000 bridova) → 1,52 ×.
- Skupovi bridova bez ciklusa čine **matroid** (manji skup uvijek se može proširiti bridom iz većeg) → pohlepa "najlakše prvo" optimalna je na svakom matroidu.
- Pohlepa **ne radi** za najdulji jednostavni put (NP-težak — ruksak je bio lak samo jer je DAG) ni kad cijena brida ovisi o tome tko je već spojen.
- Primjene: grupiranje (single-linkage = redoslijed Kruskalovih spajanja; SciPy koristi Prima s poljem), segmentacija slike (Felzenszwalb–Huttenlocher), dizajn mreža.
- **Veza s 11. tjednom — 2-MST heuristika za TSP:** MST → udvostruči bridove (Eulerizacija) → Eulerov krug → preskoči već posjećene vrhove → Hamiltonov ciklus najviše 2 × optimum (uz nejednakost trokuta).

Odjeljak 6 — Sažetak

Četiri problema, jedna jednadžba i jedna iznimka

Svaki algoritam dodaje jedan sastojak prethodnom — i daje svoj odgovor na okvirno pitanje.

Bellman–Ford → DAG / ruksak → Bellmanova jednadžba → A* → Prim

Obećanje, cijena, pretpostavka — i odgovor na okvirno pitanje

problem	metoda	dodani sastojak	složenost	ako pretpostavka padne
najjeftiniji put s negativnim bridovima	Bellman–Ford	runde (DP po broju bridova)	$O(n \cdot m)$	negativni ciklus → nema odgovora, samo ciklus
0/1 ruksak	DAG / ruksak	graf stanja + topološki red	$O(n \cdot W)$	velike težine → graf prevelik (aproksimacija)
najjeftiniji put do jednog cilja	A*	procjena ostatka h	≤ Dijkstra; ovisi o h	h precjenjuje → skuplji put
MST	Prim	ključ = sam brid; svojstvo reza	$O(m \log n)$ / $O(n^2)$	cijena brida ovisi o drugima → pohlepa gubi jamstvo

Kada smijemo izabrati idući korak bez računanja ostatka? (što metoda zna o ostatku → što joj to dopušta)

- ništa → Bellman–Ford: izračunaj ostatak točno, u rundama
- ostatak nema povratnih bridova (DAG) → topološki red / ruksak: svaki vrh jednom, jedan prolaz
- ostatak ≥ 0 → Dijkstra: najmanju poznatu cijenu odmah proglasi konačnom
- donja granica ostatka → A*: usmjeri pretragu na jedan cilj
- zamjena (svojstvo reza) čini ostatak nebitnim → Kruskal / Prim: uzmi najlakši siguran brid i ne vraćaj se

Zajedničke ideje: **aditivnost** → Bellmanova jednadžba · relaksacija · **redoslijed** određuje algoritam · **brojač u stanju** čini graf podproblema acikličnim (k kod BF-a, i kod ruksaka, baterija kod A*) · **nagib / teleskopiranje** (potencijal → procjena) · **isti kostur s redom, drugi ključ** (Dijkstra → A* → Prim).